Decomposable Draft

Thomas Chatain, Neha Rino

June 13, 2022

Abstract

The subject of this paper is to study conformance checking for timed models, that is, process models that consider both the sequence of events in a process as well as the timestamps at which each event is recorded. Time-aware process mining is a growing subfield of research, and as tools that seek to discover timing related properties in processes develop, so to does the need for conformance checking techniques that can tackle time constraints and provide insightful quality measures for time-aware process models. In particular, one of the most useful conformance artefacts is the alignment, that is, finding the minimal changes necessary to correct a new observation to conform to a process model. From alignments, one can derive measures of fitness, precision, and generalisation of a process model, which are crucial to analysing its performance. We develop three metrics by which to compare timed processes, and seek to solve the alignment problem, that is, the problem of calculating the closest trace in the language of a timed model to a particular observed event trace, where each such metric devised provides a new notion of closeness. For the first metric, we have an algorithm that aligns traces to any extended free choice time petri net, with time complexity linear in the size of the model and it's transition set. For the second metric, we have a general approach to solve the alignment problem using the simplex algorithm, which has polynomial average time complexity and exponential worst case time complexity, and for a subclass of models we have a quadratic time algorithm. For the third metric we have an approach that aligns words with linear models, and this has time complexity linear in the size of the word, and we provide simple python implementations of the same.

1 Introduction

1.1 Conformance Checking and Alignments

Process mining studies vast systems through their event logs, and seeks to extract meaningful ways to model the underlying patterns or processes that govern the behaviour of the system, sometimes in order to better understand the system, or as is often the case, predict future behaviour [**?**]. Once such a process model is obtained, through machine learning or other related techniques, it is

natural to ask how one is sure the obtained model is a reasonable approximation of the system's behaviour at all, especially given the lack of explainability in the blackbox approach ML takes to producing solutions. This is where conformance checking comes into the picture, as it is the art of judging the performance of a process model by comparing its behaviour to the observed behaviour of the system, that is, relating modelled and recorded behaviour of a process to each other, independent of the origin of the model [**?**]. Observed behaviour usually comes in the form the traces of an event log, as a sequence of events that often belong to multiple distinct interleaved instances of processes of the system. On the other hand, we have process models, that were either obtained automatically through mining, or were constructed manually, and act a blueprint to describe what the underlying processes of any given system are supposed to look like. Measures of how well a model reflects system behaviour include fitness, precision, generalisation, and simplicity. We often do not want the system to always precisely generate any and all possible future system behaviour as this can give trivial models accepting for example all execution strings, but on the other hand, neither should they simply regurgitate the event log and accept no new system behaviours, a phenomenon known as overfitting, or not generalising properly. What is often much more useful is a process model that can, up to some small error factor, approximate any reasonable future system behaviour quite closely. This flexible boundary between what is expected and what is possible is precisely what makes a model useful.

In Aryan Adriansyah's seminal thesis [**?**] , we obtain the notion of an alignment, that is, the minimal series of corrections needed to transform an observed event trace into the execution of the process model that most closely mimics it. It is often seen as an execution of the synchronised product of the process model and a *trace model* ie a simple model that captures exactly one event log trace. This gives us a series of edits, usually insertions or deletions, that can transform a model run into the observed run. For our purposes we will use the term alignment to mean the optimal run of the model that aligns closest to the observation, and how to transform this optimal execution to the observed one will be addressed separately as a matter of distance computation over a set of edit moves. Alignments thereby help pinpoint exactly where inevitable deviations from expected behaviour occur, and the more distant the aligning word of a model is to its observed trace, the worse the model is at reflecting real life behaviour. This can either be used as a tool to help repair the process model to more accurately reflect the observation, or, it can be used to repair the log, or even issue feedback on the behaviour of the underlying system to help it conform to the model better, depending on which components of the operation are the most trustworthy.

1.2 Time-aware Processes

Process models are often formalised using the notion of a Petri net, a standard object used to describe distributed systems. They both offer a graphical means by which to represent a complex system, and a formal semantics for their

execution, which allows one to mathematically analyse the same. They are useful when trying to express dependencies in seemingly parallel parts of the system, the graph structure allowing for branching (i.e. choice), convergence (i.e. synchronisation), and cycles (i.e iteration) in system behaviour. Assuming the event logs are a list of words over a finite alphabet (the set of possible discrete events), the problem of calculating the alignment has been extensively studied [**?**] [**?**]. The notion of distance used on these words used is usually either Hamming distance or Levenshtein's edit distance. Given that Petri net processes already do have certain temporal characteristics, such as a (partial) sense of what happens before and after, and a notion of causal dependency between events, it is natural to want to study explicitly timed systems, that is, systems that not only retain the sequence of events that occur, but the timestamps that show when each event happens. By considering events along with their timestamps when mining processes, observations regarding complex time constraints can be obtained, such as the minimum delay between two events, or the maximum duration the system takes to converge upon a state, or meeting deadlines, all of which are highly relevant in real world applications [**?**] [**?**] [**?**]. In addition, one may want to predict the timestamps of processes [**?**]. In order to formalise such timed process models, we can use Time Petri nets, that are Petri nets augmented with the ability to record and check the duration it takes to fire a transition once enabled, using which one can impose certain constraints on the relationships between the timestamps of different events.

So far in the study of the alignment problem for process models, we find one notable gap. The process models concerned often do not consider event timestamps, and constraints imposed upon them. Time-aware process mining seeks to study both what sort of underlying processes govern system behaviour, and what sort of time constraints they can impose on when certain events can occur [**?**] [**?**] [**?**]. The need for such a setting is clear, as specifying and verifying temporal properties, and concrete time constraints regarding the durations of events can provide critical information about a system's behaviour. As time-aware process mining grows popular, new quality measures and conformance checking techniques must be developed that are sensitive to temporal constraints. For this, crucially, one needs to define distance functions that can meaningfully compare and separate different time processes. This paper seeks to provide a framework by which to do the very same.

1.3 Three Distances, and Algorithms to Align them

In this paper, we propose three different distance functions over timed words, corresponding to only editing one point in a word at a time (stamp edits), forcing every edit in a word to ripple out and displace all of its causal successors (delay edits), or a combination of the two (mixed edits). The first is essentially Manhattan distance, and hence is extremely practical, and has a quadratic time algorithm in the case of a structurally restricted class of models (linear causal processes), that is, process models whose runs look like straight lines with no branching or concurrency. For all other types of models, we present an

encoding in simplex that solves the problem, albeit with exponential worst-time complexity and good efficiency in practice. The second distance function utilises the structure of the process model to study distances between time processes, and we present a straightforward algorithm for solving the alignment in this case, whose time complexity is linear in the size of the causal process and the transition set of the model. This setting may be of slightly limited scope as it is only relevant if one always knows that every single error will have knock-on effects, such as in event logs that only perceive relative durations but output absolute timestamps. The third is the most versatile, and the most interesting for our purposes. In this setting, edits can either be stamp moves or delay moves, allowing one to choose whether to let an edit stay local or affect the rest of the process, depending on which would be cheaper. We present a linear time algorithm for solving the alignment problem in this setting, but once again only for process models with linear causal processes as in the first setting.

2 Preliminaries

2.1 Timed Process Models

We represent events as timed letters, that is of the form (a, t) where $a \in \Sigma$ is the name of the action taken during an event, and t denotes the time at which said action was taken. An event trace is then a sequence of events of the form $\gamma \in (\Sigma \times \mathbb{R}^+)^*$, seen as a timed word. For the context of this paper, we will often ignore the untimed parts of timed words, that is, the projection onto Σ^*, leaving just the time-series of the timed words, a sequence belonging to $\mathbb{R}^{+*}$.

The timed process model that attempts to capture the behaviour of all observed event traces recorded from the system is represented here as a labelled time Petri net.

Definition 1 (Time Petri Net)**.** *A labelled time Petri net is a tuple* $N = (P, T, F, SI, \lambda, M_0, M_f)$*, where*

1. P *is a set of places*
2. T *is a set of transitions*
3. $P \cap T = \emptyset$
4. $F \subseteq (P \times T) \cup (T \times P)$ *is a flow relation*
5. $SI : T \to \mathbb{I}$ *is the static interval function, where* $SI(t) = (Eft(t), Lft(t))$ *such that* Eft *stands for earliest firing time, and* Lft *for latest firing time.*
6. $\lambda : T \to A$ *is the labelling function, labelling transitions with actions*
7. $M_0 : P \to \mathbb{N}$ *is the initial marking of* N
8. $M_f : P \to \mathbb{N}$ *is the final marking of* N

The semantics of a time Petri net are intuitive, if a bit involved. Given a transition $t \in T$ we define the pre-set of t as ${}^\bullet t = \{p \in P | (p,t) \in F\}$ and its post-set similarly is defined as $t^\bullet = \{p \in P | (t,p) \in F\}$, just as it is defined for untimed Petri nets (the presets and post-sets of places are defined similarly). A transition t of a time Petri net is *enabled* at marking M iff $\forall p \in {}^\bullet t : M(p) > 0$. The set of all enabled transitions at a marking M is denoted by *Enabled(M)*.

This formalism begins to differ from untimed Petri nets with the notion of states and clock functions. The current marking is not sufficient to define the state of the whole system, as the time elapsed since a transition was enabled is crucial information to define the execution of the system. Hence, a *state* of a time Petri net $N = (P, T, F, SI, M_0)$ is a pair $S = (M, I)$, where M is a marking of N and $I : Enabled(M) \to \mathbb{R}^+$ is called the *clock function*. The initial state is therefore $(M_0, \mathbf{0})$, where $\mathbf{0}$ is the zero function.

A transition t is fireable from state $S = (M, I)$ after delay $\theta \in \mathbb{T}$ iff t is enabled at M, and updating the clock function to increment by delay θ will keep t's new clock value $I(t) + \theta$ in the range $[Eft(t), Lft(t)]$as determined by the static interval function. This is also referred to as t firing at time $I(t) + \theta$ at state (M, I).

Once t is fired, the marking is updated to M' as in the untimed case (subtracting one token from each preplace, and adding one token to each postplace), and the clock function I is updated I' by increasing clock values by θ for any transition that was enabled at both M and M', setting clock values to 0 for any transitions newly enabled by M' that were not enabled at M, and leaving clock values undefined for any transitions that are not currently enabled. This is denoted by the $(M, I)[t\rangle(M', I')$

An important feature of time Petri nets is the notion of urgency, that is, if a transition has been enabled then it must either be disabled (lose some tokens from its preset due to another transition firing) or be fired before the latest firing time lapses. It cannot be allowed to simply *time-out*, i.e, waiting, unfired, throughout the period of time is it capable of firing, and then being disabled just by having the Lft lapse, that is if t is enabled at marking M and has clock value $Lft(t)$, it must fire, or another transition must fire at the same instant disabling t.

Now, we want a notion of a valid execution of the model, that is, valid instances of processes the model wishes to recognise. This is why we equip our time Petri nets with a notion of a final state, or in this case, a final marking. A valid execution begins at the initial marking, and fires a sequence of transitions (representing a series of activities occuring at certain times) and the updated marking at the end of the firing sequence being M_f, with any clock function I.

Definition 2 (Language of a time Petri net)**.** *A word $w = (a_0, a_1, \dots a_n) \in A*$ is in the language of the labelled time Petri net $\mathcal{L}(N)$ if there is a firable sequence of transitions $(t_0, t_1 \dots t_n) \in T^*$ such that $\lambda((t_0, t_1, \dots, t_n)) = w$ and they transform the initial marking into the final one, that is, for some clock function I*

$$(M_0, \mathbf{0})[t_0, t_1, \dots t_n\rangle(M_f, I)$$

Example 1.

Now we will want to build some vocabulary to help us talk about individual executions of timed words on time Petri nets. These correspond to runs in nondeterministic automata, but in this setting every configuration is a set of token-filled places, so to represent a particular execution we use what we define below as causal processes.

Definition 3 (Causal Net)**.** *A causal net $CN = (B, E, G)$ is a finitary, acyclic net where*

$$\forall b \in B : |b^\bullet| \leq 1 \wedge |{}^\bullet b| \leq 1.$$

This is the ground upon which a process of the time Petri net is played. It can be viewed as the original Petri net itself, but every time a place is revisited, it is copied afresh to ensure that the execution only ever moves forward. Hence, multiple elements of B or E map to the same element in P or T respectively, a notion that is expressed via the map p defined below.

Definition 4 (Homomorphism)**.** *Let $N = (P, T, F, SI, \lambda, M_0, M_f)$ be a time Petri net and $CN = (B, E, G)$ be a causal net. A mapping $p : B \cup E \to P \cup T$ is a homomorphism if*

1. $p(B) \subseteq P$
2. $p(E) \subseteq T$
3. *$\forall e \in E$ the restriction of p to ${}^\bullet e$ is a bijection between ${}^\bullet e$ and ${}^\bullet p(e)$ and the restriction of p to $e^\bullet$ is a bijection between $e^\bullet$ and $p(e)^\bullet$ and the restriction of p to $Min(CN)$ is a bijection between $Min(CN)$ and M_0.*

Definition 5 (Causal Process)**.** *A causal process of a TPN is a pair (CN, p) where CN is a causal net and p is a homomorphism from CN to TPN.*

Using p, elements of CN are identified with their corresponding *originals* in the time Petri net, and as a result, any causal process corresponds uniquely to an untimed run on the untimed version of a given time Petri net.

2.2 Linear Causal Processes, a relevant class

We take a moment to discuss a structurally restricted class of processes, that is, causal processes whose graphical structure is that of a straight line. These reflect the executions of time Petri nets that do not have any branching points ($e \in E : |e^\bullet| > 1$) and naturally as a consequence, no points of synchrony ($e \in E : |{}^\bullet e| > 1$). This loses a significant amount of expressive power, and essentially reflects the executions of automata, allowing for exclusive branching ($p \in P : |p^\bullet| > 1$) and iteration (that is, cycles in the model) only. It loses the ability to capture properties of a concurrent nature, as the run never splits into more than one token. On the other hand, studying the language of this restricted class and the alignment problem over it becomes substantially simpler, so to begin with, this is a significantly more tractable class of time processes.

Example 2.

Example 3.

So, to summarize, there is a class of process models that have linear underlying causal processes. These time Petri nets have no branching or joins at transition nodes, but may have branches, joins, and loops on place nodes. Their causal processes are hence simple linear paths through the graph of the net, and their valid timing functions and words in their language shall often be viewed as simple linear traces, indexed by integers. The G relation becomes a total order, which makes causal dependencies simple and predictable, like a line of dominoes. There is a tradeoff between the expressiveness of this model and how much simpler they are to analyse and align, but in some cases we start with this simple setting to help motivate how to solve the alignment problem in a more general setting.

2.3 Timed Processes

Now, in order to study *timed* executions, we want to be able add a timing function to the causal process we defined above, thereby allowing us to record when different transitions are taken. The following definitions, properties and theorems in this subsection cover the material developed in Aura and Lilius' article on Time Processes [**?**].

Definition 6 (Timing Function)**.** *A timing function $\tau : E \to \mathbb{T}$ is a function from events of a causal process into time values.*

Now, given this function, we try to see how the process unfolds. First the initial events, that is, those enabled at $Min(CN)$ happen, and as each event occurs, new events are enabled and disabled. In order to keep track of this, we first define the auxiliary Cut function that represents the effect of having all the events in a subset fire simultaneously (we only use Cut on sets of events that can concurrently fire), as defined below.

$$Cut(E') = (E'^{\bullet} \cup Min(NS)) \setminus {}^{\bullet}E'.$$

Now we can define the *time of enabling* for a transition t of a TPN in a set of conditions B' of its causal process as

$$TOE(B', t) = max(\{\tau({}^{\bullet}b) | b \in B' \setminus Min(CN) \wedge p(b) \in {}^{\bullet}t\} \cup \{0\})$$

We must now verify that this timing function does indeed represent a valid execution that obeys all the static interval constraints, as below :

Definition 7 (Valid Timing)**.** *A timing function τ is a valid timing of the causal process iff*

$$\forall e \in E : \tau(e) \geq TOE({}^{\bullet}e, p(e)) + Eft(p(e))$$

and

$$\forall e \in E : \forall t \in Enabled(p(C_e)) : \tau(e) \leq TOE(C_e, t) + Lft(t)$$

where $C_e = Cut(Earlier(e))$.

Here, checking every element of C_e might seem unnecessary but in certain nets, complex dependencies between transitions can cause causally unrelated transitions to force transitions to fire or be disabled, due to urgency. This phenomenon is known as confusion, and hence to guard against this, a timing function must check that it never leaves a section of the process behind, completing the firing or disabling of every event in $Earlier(e)$ before it can reason about the fireability of e.

Definition 8 (Time Process)**.** *A time process of a TPN is a triple* (CN, p, τ) *where* τ *is a valid timing of* (CN, p) *which is a causal process of the TPN.*

A few last notions here will assist us when we consider a simpler class of time Petri nets, that allow for much easier validity checking.

Definition 9 (Extended Free Choice)**.** *A time Petri net is extended free choice iff for all two transitions* t *and* t', ${}^\bullet t \cap {}^\bullet t' \neq \emptyset$ *implies* ${}^\bullet t = {}^\bullet t'$.

This class of time Petri nets ensure that the net is *confusion-free*, that is, causally unrelated events cannot affect the fireability of other events. This means that in order to check for the validity of a timing function, checking all of C_e as before is no longer necessary.

How do we construct a valid timing function on the fly? Given a partial time process of a TPN, we want to be able to study the effect of firing particular transitions amongst the set of transitions currently enabled. In order to do so, we notice the following class of transitions.

A transition t is a *choice* at $B' \subseteq B$ iff B' is a co-set that maps injectively to places and ${}^\bullet t = p(B')$, and a choice is an *extension* of the process iff $B' \subseteq Cut(E)$.

These transitions reflect exactly the transitions enabled at a particular moment in the evolution of the process. Either they must be fired or disabled, by the urgency condition.

Given the above, Aura and Lilius characterise a method by which one can build partial time processes inductively, building forward while maintaining the validity of the timing function, denoted as keeping the process complete with respect to the timing function :

Definition 10. *A causal process* (CN, p) *of a time Petri Net where* $CN = (B, E, G)$ *is said to be complete with respect to a timing function* τ *iff for every extension transition* t *of the process,*

$$\max\{\tau(e) | e \in E\} \leq TOE(Cut(E), t) + Lft(t).$$

Now given this, the following theorem due to Aura and Lilius [**?**] holds :

Theorem 1. *Let N be an extended free choice time Petri net and (CN, p) a causal process of N, where $CN = (B, E, G)$. A timing function τ is valid iff the following criteria hold :*

1. $\forall e \in E : Eft(p(e)) \leq \tau(e) - TOE({}^\bullet e, p(e))$
2. $\tau(e) - TOE({}^\bullet e, p(e)) \leq \min\{Lft(t) | {}^\bullet t = {}^\bullet p(e)\}$
3. *(CN, p) is complete with respect to τ, i.e, for every extension transition t of the process,*

$$\max\{\tau(e) | e \in E\} \leq TOE(Cut(E), t) + Lft(t)$$

3 The Alignment Problem in Timed Settings

The alignment problem, given a trace from an event log and a process model, involves finding a valid execution of the process model that is closest to the trace under some metric.

Definition 11 (The Alignment Problem)**.** *Given a process model N denoted by a Time Petri Net and a timed trace σ we wish to find a timed word $\gamma \in \mathcal{L}(N)$ such that*

$$d(\sigma, \gamma) = \min_{x \in \mathcal{L}(N)} d(\sigma, x)$$

for some distance function d on timed words.

There are two issues that spring to mind here, that are in a way preliminary problems to the task at hand. The first is the question of how the untimed part of the word (the letters, the underlying causal process) should interact with the timed part.

One might imagine one way to address this problem is by the following heuristic : First align the untimed part of the trace with the untimed version of the petri net, and then using the partial order one obtains on the closest untimed trace, proceeding to then try to align the time sequences. While this does neatly separate the problem into the untimed case and a case involving only time sequences, it might not always be very practical. If instead, we decide to view this as a cost minimization problem, given one fixed cost for editing the action names themselves, another fixed cost for editing the time sequences, then the above heuristic is like assigning a significantly higher cost to action edits than time edits, and in some cases this can be an exaggeration. Much like in the untimed case, we may even want to adopt a discounted cost function, ie a cost function that decays as one progresses further in the word, causing earlier errors to be significantly more costly than later ones.

The second is a subtler question, regarding nondeterminism in the process model. Especially in the case of silent transitions, but even otherwise, sometimes a given word does not neatly map onto a single causal process. Hence, if we

try to found our study of alignments on the idea of producing similar looking causal processes, a single causal process must be chosen for σ first.

For the purposes of this paper, we ignore the untimed parts of the words, and the problem of nondeterminism, i.e we assume the underlying causal processes and event labels match, and simply the timing functions on them differ in the words whose distance we wish to measure and minimize. This ensures that we can compare the timing constraints on analogous parts of the process, and hence reason meaningfully about deviances from the model that are purely based on the timing of the process. This gives us a slightly different problem than the general setting of the alignment problem, that focuses purely on timing properties, that is, a purely timed alignment problem, as defined below :

Definition 12 (Purely Timed Alignment Problem)**.**

For the rest of this paper we restrict our attention to the purely timed alignment problem, for each of the distances we shall define below.

3.1 Two Moves, Two Representations

Now we come to the problem of deciding how to compare two time sequences, and quantify how close they are to each other. As mentioned above, henceforth we assume we are comparing time sequences whose underlying causal processes are the same, in order to meaningfully compare the relationships between their delays.

Much like Levenshtein's edit distance, popularly used in the untimed case of the alignment problem, we view the definition of these distances as an exercise in cost minimisation over the set of all transformations between two words. In order to formalise the same, we need to define what the valid moves of such a transformation could be. We define *moves* as functions that map one timing function over (CN, p) to another. What sort of functions are useful notions of transformation on a timed system?

Let us consider the following example :

Example 4. Move Intuition

Based on the above example, we naturally arrive at two types of moves.

We define a *stamp move* as a move that translates the timing function only at a point, ie, that edits a particular element of the timestamp series τ. Calculating d_t between two traces can be viewed as a cost minimization process in aligning the traces using only stamp moves.

Definition 13 (Stamp Move)**.** *Given a timing function $\gamma : E \to \mathbb{R}$, formally, we define this as :*

$\forall x \in \mathbb{R}, e \in E : stamp(\gamma, x, e) = \gamma'$ *where*

$$\forall e' \in E : \gamma'(e') = \begin{cases} \gamma(e') + x & e' = e \\ \gamma(e') & otherwise \end{cases}$$

The next type of move we describe is the more novel and interesting *delay move*. By formulating this type of edit move, we sought to leverage the structure of the process model itself, by reflecting the causal relationships the pre-order G causes. If an event is G-reachable from another, that means the first event is strictly in the causal history of the second. One can conceive of situations where changing the timestamp of such a causal predecessor has consequences for all of its causal descendents, while leaving any causally unrelated (i.e, non-G-reachable) events undisturbed. This is the motivation behind a delay edit. A stamp move is purely local, such that when a stamp edit occurs at e its immediate successor events shift their relationship with e, thereby ensuring that the change in e's timestamp does not derail the rest of the system. On the other hand, any delay move at e will preserve relative relations in the future, at the cost of shifting the timestamp of every causal descendent of e by the same amount. Alternatively, this formulation of a move can be seen as ensuring one need not pay multiple costs for translating an entire causally linked branch of the process by a certain amount, instead editing (and paying) just once, judiciously early on, and reaping its rewards later without cost.

So, we define a *delay move* as a move that translates the timing function at a point and all of its G-successors.

Definition 14 (Delay Move)**.** *Given a timing function $\gamma : E \to \mathbb{R}^n$, we define a delay move applied to it as follows :*

$\forall x \in \mathbb{R}, e \in E : delay(\gamma, x, e) = \gamma'$ *where*

$$\forall e' \in E : \gamma'(e') = \begin{cases} \gamma(e') + x & e' \geq_G e \\ \gamma(e') & otherwise \end{cases}$$

We will also define a third kind of move, although really it is simply convenient notation for a common combination of the previous two rather than a new mechanism of transformation.

Definition 15 (Mixed Move)**.** *We define mixed moves, that denote doing a stamp move and a delay move at the same position in the word. Let $Moves(CN, p) = \{(s, d, e) | s, d \in \mathbb{R}, e \in E\}$ be the set of possible mixed moves over any trace with underlying causal process (CN, p), with $Stamp(CN, p) = \{(s, 0, e) | s \in \mathbb{R}, e \in E\}$ and $Delay(CN, p) = \{(0, d, e) | d \in \mathbb{R}, e \in E\}$ being the subsets of moves corresponding only to pure stamps or pure delays, respectively. We define their effect as follows :*

$$(s, d, e)(\gamma) = stamp(delay(\gamma, d, e), s, e)$$

Given any move $m \in Moves(CN, p)$ we define the function $cost : Moves(CN, p) \to \mathbb{R}^+$ that returns the cost of the move. The cost of a mixed move (s, d, e) is the sum of the cost of the stamp and delay moves it's made up of, $|s| + |d|$.

We say a sequence of moves $m_1 m_2 \dots m_n = m \in Moves(CN, p)^*$ *aligns* τ_1 to τ_2 if

$$m(\tau_1) = m_n(\dots(m_2(m_1(\tau_1))\dots) = \tau_2$$

Now, armed with these types of moves, three natural notions of distance can be constructed.

Definition 16 (Stamp Only Distance : d_t)**.** *Given any two timing functions τ_1, τ_2 over the same causal process (CN, p), we define the stamp-only distance d_t as follows :*

$$d_t(\tau_1, \tau_2) = \min_{m \in Stamp(CN,p)^*, m(\tau_1)=\tau_2} cost(m)$$

Definition 17 (Delay Only Distance : d_θ)**.** *Given any two timing functions τ_1, τ_2 over the same causal process (CN, p), we define the delay-only distance d_θ as follows :*

$$d_t(\tau_1, \tau_2) = \min_{m \in Delay(CN,p)^*, m(\tau_1)=\tau_2} cost(m)$$

Definition 18 (Mixed Moves Distance : d_N)**.** *And thirdly, given any two timing functions τ_1, τ_2 over the same causal process (CN, p), we define the mixed move distance d_N as follows :*

$$d_t(\tau_1, \tau_2) = \min_{m \in Moves(CN,p)^*, m(\tau_1)=\tau_2} cost(m)$$

With delay edits, which constitute a new way of thinking about how a time-series can be transformed, comes a new perspective with which we can view timing functions over causal processes. There is of course the standard definition, $\tau : E \to \mathbb{R}^+$ that assigns to each event a timestamp that records exactly when the event occurs.

Instead, thinking along the lines of delays and durations between events occuring, we will often benefit from considering the following representation when speaking about delay moves, and so we define a way by which to view a timed word not in terms of its absolute timestamps, but by the delays between relevant timestamps.

Definition 19 (Flow Function)**.** *Given a causal process (CN, p) over N, where $CN = (B, E, G)$, and a (not necessarily valid) timing function $\tau : E \to \mathbb{R}^+$, we first define the flow function of τ, $f_\tau : E \to \mathbb{R}^+$ such that*

$$f_\tau(e) = \begin{cases} \tau(e) & {}^{\bullet\bullet}e = \emptyset \\ \tau(e) - \tau(e') & e' \in {}^{\bullet\bullet}e, \tau(e') = \max_{e'' \in {}^{\bullet\bullet}e} \{\tau(e'')\} \cup \{0\} \end{cases}$$

The flow function thereby is a dual representation of timing functions, much like the relationship between graphs and line graphs, as if a timing function traditionally labels its transition nodes with timestamps, the flow function labels edges leading up to transitions with the duration since said transition was enabled.

Example 5.

Note that e' as defined here is the latest causal predecessor of e, that is, it is the firing of e' that enabled e. Hence, f_τ as defined produces exactly the time durations that the guards of each transition in the model checks, ie the clock function values during the run.

As defined, we see that if a word is in the language of the model, then its f function maps events to values that lie within the constraint that the event's corresponding transition demands, that is, it is perfectly aligned with the model with distance 0. This condition is unfortunately not sufficient though, due to urgency concerns, but it is quite close to the exact condition necessary for a word to be in the language of a time Petri net.

Also note that given the underlying causal process and the resulting f_τ, we can reconstruct τ quite straightforwardly as

$$\forall e \in E : \tau(e) = \sum_{e' \leq_G e} f_\tau(e')$$

With a new representation, it is natural to study how the transformations we have defined earlier on timing functions transform their respective flow functions. A stamp move at an event e of value $+x$ will increase the duration between e and its immediate enabling predecessor, while shrinking the duration between e and all of its immediate enabled successors by an equal amount, thereby moving e by x and compensating appropriately so none of the future timestamps change at all. On the other hand, a delay move at e will increase the duration between e and its immediate enabling predecessor, keeping all other durations constant, thereby ensuring e and all of its causal descendents have their timestamps shifted by x. So both these moves are much more local, effectively perturbing only the duration attached to e or its immediate enabled successors, while in the timing function representation moves at a position could have consequences well beyond its immediate neighbours. This gives us the following lemma :

Lemma 2. *Given a flow function f_{τ_1} and a mixed move (s, d, e), the consequence of performing the move is the flow function defined below, where $\tau_2 = (s, d, e)\tau_1$.*

$$f_{\tau_2}(e') = \begin{cases} f_{\tau_1}(e') + s + d & e' = e \\ f_{\tau_1}(e') - s & e \in {}^{\bullet\bullet}e', \tau(e) = \max\limits_{e'' \in {}^{\bullet\bullet}e'} \{\tau_2(e'')\} \end{cases}$$

Lemma 3 (Stamp only distance : d_t)**.** *d_t as defined above is equivalent to the Manhattan distance or taxicab distance between two points of $\mathbb{R}^n$, where $|E| = n$, say $\mathbf{t} = (t_1, t_2, \ldots t_n)$ and $\mathbf{s} = (s_1, s_2, \ldots s_n)$, defined as*

$$d_t(\mathbf{t}, \mathbf{s}) = \sum_{i=1}^{n} |t_i - s_i|$$

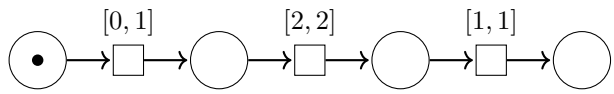
Proof. The way we see this is as follows. Given a causal process $CN = (B, E, G)$ and two timing functions (not necessarily constituting valid timings) τ_1, τ_2, let

$$d_t(\tau_1, \tau_2) = \sum_{e \in E} |\tau_1(e) - \tau_2(e)|$$

This is both achievable by stamp only moves by doing the corresponding $\tau_1(e) - \tau_2(e)$ stamp move to the second word, and is the least cost any such stamp only path can take as Manhattan distance obeys the triangle inequality, and every stamp move results in a word at exactly the same Manhattan distance from its predecessor as the cost of said stamp move.

□

Example 6. Consider the following example :



Now for this N, let the observed trace $\sigma = (0, 3, 4) \notin \mathcal{L}(N)$.

The best d_t alignment for the example in the diagram below is $\gamma_t = (1, 3, 4)$ because

$$d_t(\gamma_t, \sigma) = 1 \leq d_t((x, x+2, x+3), \sigma) = |x| + 2|x-1|, \forall x \in [0, 1]$$

Lemma 4 (Delay only distance : d_θ)**.** *d_θ as defined above based on moves, is equivalent to the following formulation directly using the flow function previously defined, as follows :*

$$d_\theta(\tau_1, \tau_2) = \sum_{e \in E} |f_{\tau_1}(e) - f_{\tau_2}(e)|$$

Proof. By noting that delay moves on a trace τ translate exactly to stamp moves on f_τ, we can deduce by the same argument as Lemma 3 that the distance function d_θ can be seen to be identical the manhattan distance on flow functions, i.e, the above formulation.

□

Example 7. Studying the model and observed trace seen in Example 6, but in the delay only setting, we see that since $\mathcal{L}(N) = \{(x, x+2, x+3) | \forall x \in [0, 1]\}$, the best d_θ alignment for the above trace and model pair is $\gamma_\theta = (0, 2, 3)$ because

$$d_\theta(\gamma_\theta, \sigma) = 1 \leq d_\theta((x, x+2, x+3), \sigma) = |x| + 1, \forall x \in [0, 1]$$

Finally we define one more possible distance function, combining the previous two concepts. Here, we allow for either editing absolute timestamps or relative time durations via delays, that is, we allow the use of either type of move.

Given two timing functions τ_1, τ_2 and their underlying causal process (CN, p), we define $d_N(\tau_1, \tau_2)$ to be the minimum total cost over all possible sequences of moves that align τ_1 to τ_2.

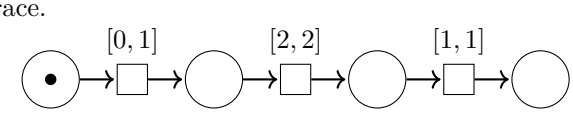
Computing this distance is no longer straightforward the way it was for the first two settings. As an example, consider the following :

Example 8. Now, coming back to the model and observed trace we studied in Example 6, but now in the mixed setting. Once again, we note that any word in the language would be of the form $(x, x+2, x+3)$ with $x \in [0, 1]$.

Now intuitively, when dealing with linear models where the ripple effect of a delay is always predictably to the right irrespective of the values of the timestamps, moves can be applied in any order, so we can assume a minimum cost move is chronological. So our first move will have to incur a minimum cost of $|x| = x$ as we try to align $(0, 3, 4)$ to our model. Now, if any part of this initial move was a delay, we would push the second component even further away from $x+2$ than it already was, and if any of the delay were negative, then the mixed move seems counterproductive at the first component as the delay and stamp moves work against each other, so let us say the first move was pure stamp. Now we move on to move two, which will again incur a minimum cost of $|1-x| = 1-x$. Now, if the stamp part of this move were positive, it would leave the third component further off from the goal $x+3$ than if all of the move was delay, and so a pure delay move seems best here. Hence, the least cost alignment is $(x, 0, 1)(0, 1-x, 2)$, with total cost 1.

All the intuitive reasoning given above will be justified in subsequent sections, and in fact distance 1 is the best we can do here even in the mixed case. Hence the aligning words here constitute the whole language $\mathcal{L}(N)$ as they are all at distance 1 from the observed trace in this setting.

Example 9. Consider the same underlying linear model, and a new observed trace.



Now for this N, let the observed trace $\sigma = (3, 4, 5) \notin \mathcal{L}(N)$.

The best d_t alignment for the example in the diagram below is $\gamma = (1, 3, 4)$ with minimum cost $d_t(\sigma, \gamma) = 4$.

The best $d_t = \theta$ alignment for the example in the diagram below is also $\gamma = (1, 3, 4)$, but this time with minimum cost $d_\theta(\sigma, \gamma) = 3$.

And lastly, the best d_N alignment for the example in the diagram below is also $\gamma = (1, 3, 4)$, the sequence of moves being simply one mixed move at the start, $m = (-1, -1, 1)$, and now with minimum cost $d_N(\sigma, \gamma) = 2 < \min\{d_t(\sigma, \gamma), d_\theta(\sigma, \gamma)\}$.

4 Results and Algorithm : Delays Only Setting

The delay only distance, d_θ, is built so as to allow local edits that do not affect global membership in the language of the time Petri net, as both the time Petri net and delay edits concern themselves only with the duration of time elapsed between a transition's enabling and its firing. This relationship is especially clear in extended free choice time Petri nets, where the only events that determine the fireability of an event are its causal history and events with the exact same preset, which would naturally disable it by firing. Hence, we present the

Algorithm 1 Naive d_θ Algorithm

Input : N, $CN = (B, E, G), p$, $\sigma : E \to \mathbb{R}^+$
Output : $\gamma : E \to \mathbb{R}^+$ such that (CN, p, γ) is a valid time process of N and $d_\theta(\gamma, \sigma) \leq d_\theta(x, \sigma)$ for all x such that (CN, p, x) is a valid time process of N

$Curr = Min(CN)$
$Enabled = \{t | {}^\bullet t \subseteq p(Curr)\}$
$FD = \{(t, eft, l, toe) | t \in Enabled, eft = Eft(t), toe = 0, l = \min_{\bullet\, t' = \bullet\, t} Lft(t')\}$
$Soon = \{(t, eft, l, toe) \in FD | l + toe$ is minimal in $FD\}$
while $E \neq \emptyset$ **do**
  if $\pi_1(Soon) \cap p(E) = \emptyset$ **then**
    REJECT : invalid causal process
    return None
  else:
    Pick $(t, eft, l, toe) \in Soon, t \in p(E)$
    $E \leftarrow E \setminus \{e | p(e) = t\}$
    $e \leftarrow p^{-1}(t)$
    $Curr \leftarrow \{Curr \setminus \{{}^\bullet e\}\} \cup \{e^\bullet\}$
    $f_\gamma(e) = \text{argmin}_{x in [eft, l]} |x - f_\sigma(e)|$
    for all $t' \in T \wedge {}^\bullet t' = {}^\bullet t$ **do**
      $Enabled \leftarrow Enabled \setminus \{t'\}$
      $FD \leftarrow FD \setminus \{(t', e', l', toe')\}$
    end for
    for all $t' \in T \wedge t'^\bullet = t^\bullet$ **do**
      $Enabled \leftarrow Enabled \cup \{t'\}$
      $eft' \leftarrow Eft(t')$
      $l' \leftarrow \min_{\bullet\, t'' = \bullet\, t'} Lft(t'')$
      $toe' \leftarrow toe + f_\gamma(e)$
      $FD \leftarrow FD \cup \{(t', eft', l', toe')\}$
    end for
    $Soon = \{(t, eft, l, toe) \in FD | l + toe$ is minimal in $FD\}$
  end if
end while
return γ

following algorithm, which minimizes the delay cost at each transition locally, while still overall ensuring membership in the language of the model.

Theorem 5. *Algorithm 1 is correct on extended free choice time petri nets, that is, its result γ has the properties :*

1. *$(CN, p, \gamma$ is a valid time process of N.*

2. *$d_\theta(\gamma, \sigma) \leq d_\theta(x, \sigma)$ for all x such that (CN, p, x) is a valid time process of N.*

Proof. By theorem 1 proved in Aura and Lilius' article [**?**], we know that in order to ensure that this is a valid time process, we need only ensure three inequalities hold as the time process γ evolves.

1. $\forall e \in E : Eft(p(e)) \leq \gamma(e) - TOE({}^\bullet e, p(e))$

   By definition $f_\gamma(e) = \gamma(e) - TOE({}^\bullet e, p(e))$, and by line 6 of the algorithm the variable eft is correctly initialised to store $Eft(p(e))$, and line 17 ensures the above inequality holds.

2. $\gamma(e) - TOE({}^\bullet e, p(e)) \leq \min\{Lft(t) | {}^\bullet t = {}^\bullet p(e)\}$

   In similar vein, line 6 also ensures that l is initialised precisely to store $\min\{Lft(t) | {}^\bullet t = {}^\bullet p(e)\}$, and line 17 again enforces the left hand side to be less than l.

3. (CN, p) is complete with respect to γ, i.e, for every extension transition t of the process,

   $$\max\{\gamma(e) | e \in E\} \leq TOE(Cut(E), t) + Lft(t)$$

   At every point when assigning a timestamp to an event, it is ensured that it belongs to *Soon*, a set defined to contain only those enabled events that have the least value of $l + toe$, or, $TOE(Cut(E), p(e)) + Lft(p(e))$. This ensures that whenever an event e is assigned a timestamp $\gamma(e)$, for all extension transitions t (which must either be causal descendents of e or already enabled when e was picked, thereby being in *Enabled*, by extended free choice) we know that $\gamma(e) \leq TOE(Cut(E), p(e)) + Lft(p(e)) \leq TOE(Cut(E), t) + Lft(t)$.

   Hence, (CN, p, γ) is a valid time process of N.

As for its optimality, we see that any other valid time process would have to obey the same inequalities, in particular for another valid timing function γ' it would have to obey

$$f_{\gamma'}(e) \in [Eft(p(e)), \min_{{}^\bullet t' = {}^\bullet p(e)} Lft(t')] = [a, b]$$

and hence at any event e where γ differs from γ' it would have lower than or equal cost for that event as

$$f_\gamma(e) = \operatorname*{argmin}_{x \in [a,b]} |f_\sigma(e) - x|$$

Hence, the algorithm is correct. □

Remark 1. *Note that Algorithm 1 runs through the events of the causal process E exactly once, and each time runs through the set of currently enabled transitions, and the total set of transitions, and hence its time complexity is linear in the size of the input $|CN|$, and the size of the transition set, i.e, $O(|E||T|)$.*

5 Results and Algorithm : Stamp Only Setting

5.1 General Bounded Time Petri Nets

The alignment problem in this case can be viewed as a problem of convex optimization, minimizing the stamp only cost, that is,

$$cost(\tau|_i) = \sum_{j=1}^{i} |\tau_j - \sigma_j|$$

over the set of possible timestamp series. What remains is to show that the space of valid runs of the model is a convex set, and this set has been studied extensively in [**?**]. In this paper, the concept of a firing domain is proposed, that is, at each configuration of the time Petri net, the set of valid time intervals in which a transition enabled at that configuration can fire. Calculating equivalence classes of the same, successively, till we find final configurations, gives a characterisation of the solution set of all valid timestamp series, and this is proved to be of the following form, which is a set of constraints defining a convex set.

Hence, we claim that using an efficient encoding of the target function using the firing domain formulation described above, the problem can be solved using the simplex algorithm, which is used to solve convex optimization problems in good time in practice, and having worst case exponential time complexity.

Theorem 6. *Given a bounded time Petri Net N, an observed trace σ, and its causal process $CN = (B, E, G)$, the alignment problem can be viewed as seeking the vector $(\gamma_1, \gamma_2, \ldots \gamma_{|E|})$ that minimizes the quantity*

$$\sum_{e \in E} |\gamma_e - \sigma_e|$$

over the set $\gamma \in \mathcal{L}(N)$.

This can be cast as a convex optimization problem, i.e,

$$\underset{\alpha - \beta \in \mathcal{C}, \alpha, \beta \geq 0}{\text{minimize}} \quad \alpha + \beta$$

Where $\mathcal{C}$ is a convex set.

Remark 2. *By the above theorem we see that the alignment problem for d_t is easily cast as a simplex instance, and can be solved by any linear optimization solver. We can thereby solve the alignment problem in polynomial average running time, but this approach does have theoretically exponential worst time complexity. This bound is not necessarily tight, but either improving this bound or proving its tightness are left for future work. While the general case as posed seems to be hard to solve efficiently, there is a subclass of process models for which the problem is significantly more tractable, and moreover, has a quadratic time solution.*

5.2 Linear Causal Processes

We are given a process model N, an observed trace σ, and its underlying linear causal process $CN = (B, E, G)$ with homomorphism p for the untimed run of σ on N.

Suppose we tried to express the cost of aligning a prefix of σ to some timing function τ on the truncation of the linear causal process up to the ith transition, call it *cost*.

$$cost(\tau|_i) = \sum_{j=1}^{i} |\tau_j - \sigma_j|$$

Now, note the following equation holds for the $(i+1)$th static interval constraint $[a, b]$:

$$\min_{\tau|_{i+1} \in \mathcal{L}^{i+1}(N)} cost(\tau|_{i+1}) =$$
$$\min_{\tau_{i+1} - \tau_i \in [a,b]} (|\tau_{i+1} - \sigma_{i+1}| + \min_{\tau|_i \in \mathcal{L}^i(N)} cost(\tau|_i))$$

This suggests that the minimal cost of aligning prefixes σ to prefixes of the model can be recursively calculated as a function of the value of the last timestamp in the alignment of a given prefix.

$$g_i(d) = \min_{\tau|_i \in \mathcal{L}^i(N) \wedge \tau_i = d} cost(\tau|_i)$$

Now, the previous equation can be expressed in terms of the g functions as follows :

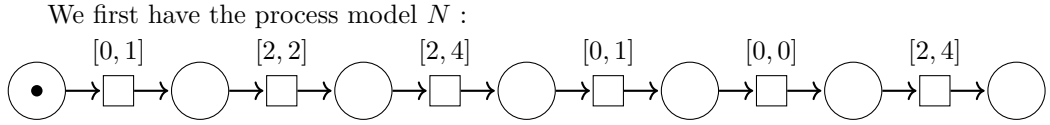
$$g_{i+1}(d) = |d - \sigma_{i+1}| + \min_{d - d' \in [a,b]} g_i(d')$$

If we can calculate the family of functions $\forall i \leq n : g_i$ efficiently, then the minimal cost for aligning σ to N is simply

$$\min_d g_n(d)$$

Example 10. We take a moment to look at an example of the computation of this family of functions we proposed by considering the following d_t-alignment problem :

We first have the process model N :



The observed trace is

$$\sigma = (1, 2, 4, 6, 6, 8)$$

Now, we begin to compute the family of functions g_i, starting with $g_1(x) = |1 - x|$ in the domain $[0, 1]$.

Now, utilising the recursion, we try to compute g_2.

The new domain is $[2, 3]$, a simple translation of the previous one because the guard on the second transition is the point $[2, 2]$. Now, we want the sum of both $|x - 2|$ and $\min_{x'-x\in[2,2]} g_1(x')$. The first of these functions, that is the contribution of the last timestamp to the error, is pictured in figure 1 in green dashed lines. The second component is pictured in black dashed lines. In this case, the non-green dashed component is simply a translation of g_1, as $\min_{x'-x\in[2,2]} g_1(x') = g_1 - 2$.

Now, let us construct g_3 similarly, by summing the contribution of the last timestamp, $|x - 4|$, to $\min_{x'-x\in[2,4]} g_2(x')$. We now have an orange segment in the second component! This is used to demarcate any additional segments not present in the previous g-function (if any). We see the effect of the min operation is simply to attach a segment with the same length as the ith static interval constraint, at the lowest point of g_{i-1}. Now we begin to see a simple way to construct these graphs visually : The next black graph is a translation of the previous blue one, with an orange line segment or the same length as the next static interval constraint inserted at the lowest point, and the next green graph is simply $|x - \sigma_i|$ in the new domain, and summing these two gives the next blue graph. Even summation is simpler than it looks, as these are all piecewise linear graphs, so you can simply increment the slopes of all the segments of the black-orange curve by one if they're to the right of σ_i, and by minus one if they're to the left, and translate the height up as needed. Clearly the orange, black and green segments of the next graph are obtained like this, and g_4 is their sum.

The procedure for constructing g_5 and g_6 are identical, and now finally we see that the minimum cost of 2 is achieved in g_6 by setting the last timestamp to 8. Now, we know the contribution of the 8 to the error is zero, so this means we can deduce the corresponding value in g_5 must have been 2 as well, achieved when the fifth timestamp was set to 6. Now 6 gives a contribution of 0 again to the error, letting us look at the preimage of 2 once again on g_4, and so on. We can backtrack in this manner to build an optimal alignment, such as in this case, $(0, 2, 5, 6, 6, 8)$.

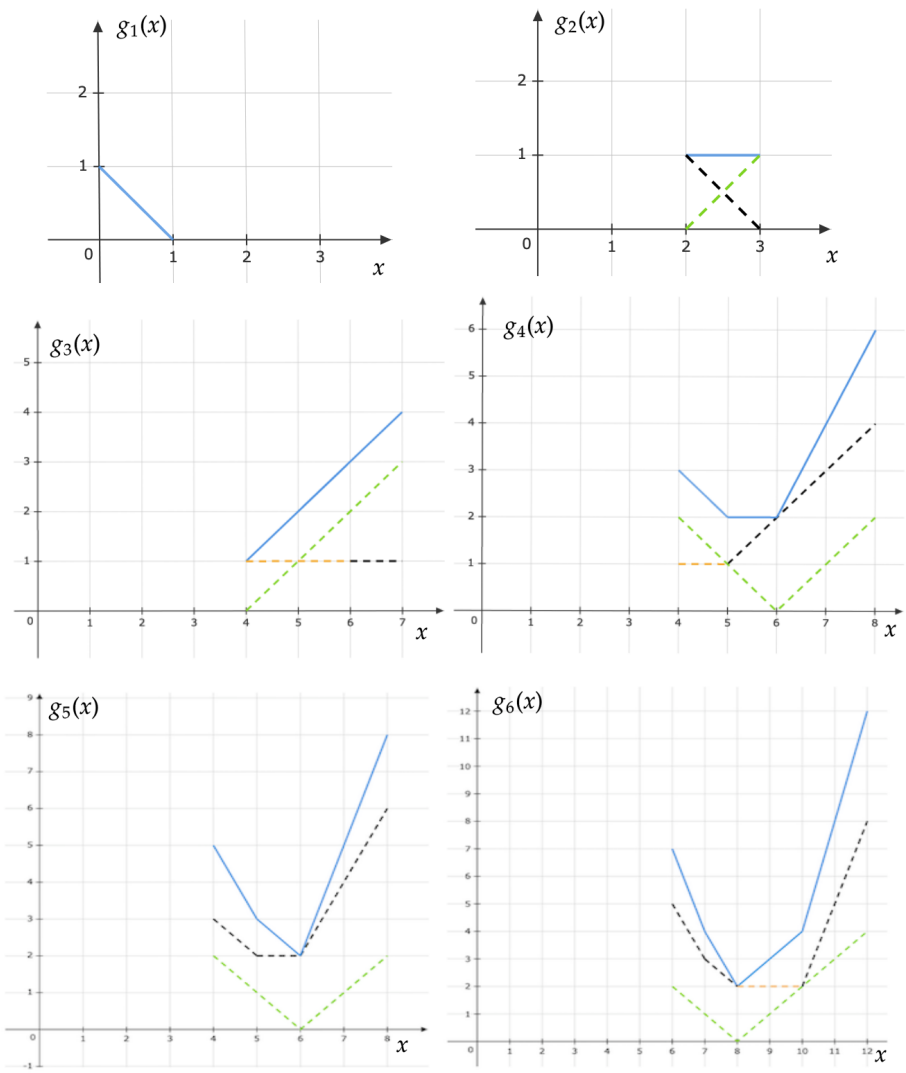


Figure 1: Visual representation of a run of the algorithm on Example 10

Algorithm 2 d_t Algorithm

Input : σ, a list of static interval constraints for the linear model SI
Output : $cost = \min_{x \in \mathcal{L}(N)} d_t(x, \sigma)$, $\gamma \in \mathcal{L}(N)$ such that $d_t(\gamma, \sigma) = cost$

Object : $graph = \{(left, slope, right) | graph[i].left = graph[i-1].right\}$
procedure StampOnlyAlgo(σ, SI)
  Initialise System
  $a \leftarrow \pi_1(SI[0])$
  $b \leftarrow \pi_2(SI[0])$
  $graph \leftarrow \{(a, 0, b)\}$
  graphAddMod($graph, \sigma[0]$)
  $graphlist \leftarrow \{graph\}$
  $i \leftarrow 1$
  $cost \leftarrow |a - \sigma[0]|$
  while $i < |sigma|$ **do**
    $a \leftarrow \pi_1(SI[i])$
    $b \leftarrow \pi_2(SI[i])$
    graphMin($graph, a, b$)
    graphAddMod($graph, \sigma[i]$)
    $graphlist.append(graph)$
    $cost \leftarrow cost + |a - \sigma[i]|$
  end while
  $i \leftarrow 0$
  $s \leftarrow graph[0].slope$
  while $graph[i].slope < 0$ **do**
    $s \leftarrow graph[i].slope$
    $cost \leftarrow cost + s * (graph[i].right - graph[i].left)$
    $i \leftarrow i + 1$
  end while
  $\gamma \leftarrow$ backTrack($\sigma, graphlist, cost, SI$)
  Return : $cost, \gamma$
end procedure

With this context, we present the following algorithm that progressively calculates the graphs of g_i each round, and claim it is both correct and efficient. The subfunction graphAddMod :

Theorem 7. *Algorithm 2 is correct, that is, given a linear causal process of a time Petri Net N, its result γ has the properties :*

1. $\gamma \in \mathcal{L}(N)$
2. $\forall x \in \mathcal{L}(N) : d_t(\gamma, \sigma) \leq d_t(x, \sigma)$

Before we prove Theorem 7, we prove a small, relevant lemma.

Algorithm 3 $g_i(d) \to g_i(d) + |d - x|$

Input : $graph$, x

Output : $newgraph$ such that if $graph$ represented the function $g(d)$ then $newgraph$ represents the function $g(d) + |d - x|$.

```
procedure graphAddMod(graph, x)
    i = 0
    while i < |graph| do
        if graph[i].right ≤ x then
            graph[i].slope ← graph[i].slope − 1
        else if graph[i].left < x ∧ graph[i].right > x then
            slope ← graph[i].slope
            high ← graph[i].right
            graph.insert(i + 1, (x, slope, high))
            graph[i].right ← x
            graph[i].slope ← slope − 1
        else
            graph[i].slope ← graph[i].slope + 1
        end if
        i ← i + 1
    end while
end procedure
```

Algorithm 4 $g_i(d) \to \min_{d-d' \in [a,b]} g_i(d')$

Input : $graph$, $[a, b]$

Output : $newgraph$ such that if $graph$ represented the function $g(d)$ then $newgraph$ represents the function $\min\limits_{d-d' \in [a,b]} g(d')$.

```
procedure graphMin(graph, a, b)
    for segment ∈ graph do
        if segment.slope < 0 then
            Translate segment to the right by a
        else if segment.slope = 0 then
            Increase length of segment by b, then translate to the right by a
        else
            Translate segment to the right by a + b
        end if
    end for
end procedure
```

Algorithm 5 Backtracking through graphs with the cost to find the best word

Input : σ, $graphlist$, $cost$, SI
Output : $\gamma \in \mathcal{L}(N)$ such that $d_t(\gamma, \sigma) = cost$
for $i = 0, i < |graphlist|$ **do**
  Construct $ylist[i]$, the list of endpoint y values of segments of $graphlist[i]$
end for
$i \leftarrow |sigma| - 1$
while $i \geq 0$ **do**
  $[a, b] = SI[i + 1]$
  Truncate $ylist[i]$ and $graphlist[i]$ to the domain $x \in [\gamma_{i+1} - b, \gamma_{i+1} - a]$.
  $k \leftarrow$ BINARYSEARCH$(ylist[i], cost)$
  $xsegment \leftarrow graphlist[i][k]$
  if $xsegment.slope = 0$ **then**
    $\gamma \leftarrow xsegment.left$
  else
    $\gamma_i \leftarrow xsegment.left + (cost - ylist[i][k].left)/xsegment.slope$
  end if
  $cost \leftarrow cost - |\gamma_i - \sigma_i|$
  $i \leftarrow i - 1$
end while

Lemma 8. *For any convex piecewise linear function g, the following function g' is also convex and piecewise linear.*

$$g'(d) = \min_{d-d' \in [a,b]} g(d')$$

Proof. For any piecewise linear convex function g, the function $g'(d) = \min_{d-d' \in [a,b]} g(d')$ is also piecewise linear. If the domain of g was $[l, h]$, the domain of g' is $[l + a, h + b]$.

Say the leftmost minimum of g (if g has multiple minima they are all on the same flat line segment) is at $m \in [l, h]$. For all $x \in [l+a, m+a]$ and $x-y \in [a, b]$ and $y \in [l, h]$ implies $y \in [x - b + a, x - a] \cap [l, h] \subseteq [l, h] \cap [l - b + am] = [l, m]$ meaning for each x, in the possible domain of y $g(y)$ is a strictly decreasing function, hence we have

$$\forall x \in [l + a, m + a] : \min_{x-y \in [a,b]} g(y) = g(x - a)$$

Now, let the rightmost minimum of g is m'. For all $x \in [m + a, m' + b]$ and $x - y \in [a, b]$ and $y \in [l, h]$ implies $y \in [x - b, x - a] \cap [l, h]$, and $[x - b, x - a] \cap [m, m'] \neq \emptyset$. Now, this means for each $x \in [m + a, m' + b]$ the domain of y has a minimum of g, so we have

$$\forall x \in [m + a, m' + b] : \min_{x-y \in [a,b]} g(y) = g(m)$$

Lastly, let $x \in [m' + b, h + b]$ and $x - y \in [a, b]$ and $y \in [l, h]$. This implies $y \in [x - b, x - a] \cap [l, h] = [m', h]$, so for each $x \in [m + a, m' + b]$ the domain of y is in a region where g is strictly increasing, so we have

$$\forall x \in [m + a, m' + b] : \min_{x-y \in [a,b]} g(y) = g(x + b)$$

So we see that for any convex piece-wise linear function g,

$$\min_{d-d' \in [a,b]} g(d') = \begin{cases} g(d - a) & l + a \leq d < m + a \\ g(m) & m + a \leq d \leq m' + b \\ g(d + b) & m' + b < d \leq h + b \end{cases}$$

Hence g' is also convex and piecewise linear, and has at most one more linear segment than g. □

We may now prove Theorem 7 as follows :

Proof. We are given a process model N, an observed trace σ, and its underlying linear causal process $CN = (B, E, G)$ with homomorphism p for the untimed run of σ on N. As discussed earlier, for linear process models, we define

$$g_i(d) = \min_{\tau|_i \in \mathcal{L}^i(N) \wedge \tau_i = d} cost(\tau|_i)$$

This family of functions is seen to obey the following recursion:

$$g_{i+1}(d) = |d - \sigma_{i+1}| + \min_{d-d' \in [a,b]} g_i(d')$$

If we can calculate the family of functions $\forall i \leq n : g_i$ efficiently, then the minimal cost for aligning σ to N is simply

$$\min_d g_n(d)$$

Hence to prove the correctness of Theorem 7 we only need show that the auxiliary subfunctions called do indeed do their job correctly, as the main function simply implements the recursion.

To this end, we first notice that the family of functions $g_i(d)$ are convex and piecewise linear. This can be seen through induction :

Firstly $g_1(d) = |d - \sigma_1|$ so the base case is covered.

Now suppose g_{i-1} is known to be convex and piecewise linear for some $i \geq 2$. By Lemma 8 we see that $\min_{d-d' \in [a_i, b_i]} g_{i-1}(d')$ is also convex and piecewise linear, and so is $|d - \sigma_i|$ and both these properties are preserved by addition, so $g_i(d)$ must also be convex and piecewise linear.

Now that we have this result, our representation of the graph as a sequence of line segments, each represented as a triple (left x endpoint, slope, right x endpoint) is justified. The y value of any graph g_i can be back calculated because we know the value of the cost of the leftmost endpoint, it represents the word obtained by firing every transition at the earliest possible time.

Now subfunction GRAPHADDMOD is adding the point σ_i if it is inside the domain of the function, and changing the slopes of the segments before σ_i by subtracting one, and of those after σ_i by adding one, which has the effect of adding the modulus component $|d - \sigma_i|$ to the previous graph.

Subfunction GRAPHMIN is even simpler, as Lemma 8 suggests all it does is translate the whole graph forward by a, and then translate the strictly increasing portion from $[m', h]$ by an addition $b - a$, adding an extra flat segment of length $b - a$ in between to keep the two portions of the function connected, and so on input $g(d), [a, b]$ it outputs $\min_{d-d' \in [a,b]} g(d')$.

Subfunction BACKTRACK takes the minimum cost obtained by the calculation of the g_n function, finds the value of d, ie γ_n for which the minimum is achieved, and subtracts the cost aligning only the last place incurs, thereby finding the minimum cost for aligning the $n - 1$ length prefix. It then proceeds to do the same iteratively for each prefix, reverse-engineering a trace γ for which the total minimum cost of aligning to σ is achieved.

□

Remark 3. *Note that the subfunctions* GRAPHMIN *and* GRAPHADDMOD *both run through the list of segments of the graph once each, and hence are linear in the sizes of their inputs, and Algorithm 2 calls each of these subfunctions once for each letter of the observed trace, each time on a graph of the size of the current prefix. So the cost calculation section of Algorithm 2 has quadratic time complexity in the size of the input.*

The backtracking subfunction BACKTRACK *runs a binary search on each stored graph in graphlist to find a trace in the language that does indeed give the minimum cost, letter by letter, and so overall takes $O(nlogn)$ time, keeping the overall time complexity of Algorithm 2 $O(n^2)$ where $n = |\sigma|$.*

6 Results and Algorithm : Mixed Setting

This setting seems to be a more versatile, but also overly forgiving notion of distance, as it assigns equal cost to delay-type errors and absolute timestamp-type errors, that is, $d_N \leq \min(d_t, d_\theta)$ so it will recognise more valid executions as "close" under a given threshold.

6.1 Linear Causal Processes

Given the non-constructive nature of the definition of d_N, it is not clear how one can efficiently calculate the distance between two fixed timed traces, as a minimal cost sequence of moves is not obvious. As before, we first study the problem over linear causal processes, as these make analysing the effects of moves significantly simpler. Before we propose an algorithm that does calculate minimal cost for linear timed traces, we first define a few properties that seem to characterise classes of well formed minimal cost runs, prove that d_N is well defined along the way, and then proceed to prove that these properties both

improve cost, and are satisfied solely by the run calculated by the algorithm we provide.

6.2 Properties of a minimal cost aligning move sequence

We wish to cull down the space of possible sequences of moves to smaller, equivalent spaces that still capture all the behaviours we're interested in, while being a little more well behaved. Hence, we introduce the following properties, which are designed intuitively to be sequences of moves that still perform the same effective transformations, and generally lower the cost of the transformation too by choosing moves wisely to avoid inefficiency. These properties (order, co-operation, and stability) have been chosen specifically to uniquely characterise the runs of the algorithm we devise to compute d_N on pairs of linear traces.

6.2.1 Order

The first property captures the order in which moves in a sequence target the positions in a linear trace. It would be good to be able to focus our attention on sequences of moves that proceed in an orderly fashion, either from left to right, doing exactly one mixed move at each position. This is intuitively reasonable as doing multiple mixed moves at the same position can only worsen the cost if it at all changes it, and assuming that mixed moves commute on linear words, it would be all the same to arrange them in order of the position it takes effect at. This gives us the following two properties.

As discussed in Section 3.1 we try to follow through the following properties in both representations, both the timing function using which we will define things, and how the same definitions reflect in the flow function.

Definition 20 (Chronology)**.** *A sequence of moves aligning γ to σ is said to be chronological if for all positions $i < j \leq n = |\gamma|$, all the moves at position i are performed before any move at position j and exactly one mixed move (where one or both components may be zero) takes place at each position, that is $\rho \in Moves(CN, p)^*$ is chronological iff*

$$\forall i \in \{1, 2, \ldots n\}, \exists s_i, d_i \in \mathbb{R} :$$
$$\rho = (s_1, d_1, 1)(s_2, d_2, 2) \ldots (s_n, d_n, n)$$

Such that

$$\forall i \in \{1, 2, \ldots n\} : \sigma_i = \gamma_i + s_i + \sum_{j=1}^{i} d_j$$

Definition 21 (Reverse Chronology)**.** *A sequence of moves aligning γ to σ is said to be reverse chronological if for all positions $i < j \leq n = |\gamma|$, all the moves at position i are performed after any move at position j and exactly one mixed move (where one or both components may be zero) takes place at each position, that is $\rho \in Moves(CN, p)^*$ is chronological iff*

$$\forall i \in \{1, 2, \dots n\}, \exists s_i, d_i \in \mathbb{R} :$$
$$\rho = (s_n, d_n, n)(s_{n-1}, d_{n-1}, n-1) \dots (s_1, d_1, 1)$$

Such that

$$\forall i \in \{1, 2, \dots n\} : \sigma_i = \gamma_i + s_i + \sum_{j=1}^{i} d_j$$

Definition 22 (Ordered)**.** *A sequence of moves aligning γ to σ is said to be ordered if it is either chronological or reverse chronological.*

Clearly, all chronological sequences have a one to one correspondence with reverse chronological sequences with exactly the same cost, the map between the two being simply reversing the sequence of moves.

In addition, in the flow function representation, the notions of chronology and reverse chronology are still somewhat preserved. A mixed move affects both the duration before and all those after the event the move is applied at, so the order in which they happen does indeed stay chronological or reverse chronological as the case may be. The only point of nuance here is that of course, a prefix of a sequence of say chronological moves is no longer has the property of having completely aligned the word up to the i-th element of the flow vector while leaving the rest unchanged, as the last edit may have perturbed the (i+1)st element.

But, the upshot is, in a reverse chronological sequence of moves, the prefix of the sequence that stops at the ith event, combined with the stamp component of the mixed move at position (i-1), completely aligns the flow function for the suffix starting at position i. Reverse chronological runs do not have an equivalent partial completion property in the timing function setting, but due to how local the moves end up being in the flow function setting, we obtain it here.

Note : For simplicity, we always assume the move played on the last position of a trace henceforth is a pure delay. Even if part of it were stamp, it would not affect the flow or timing representation any differently, so this is safe to assume.

Example 11. Given $\gamma = (1, 1, 2, 4, 5)$ and $\sigma = (1, 2, 2.5, 4.2, 5)$, an example of a cost 5.3 non-chronological sequence of moves aligning them is as shown below.

$$\gamma \xmapsto{(-1,0,1)} (0, 1, 2, 4, 5) \xmapsto{(0,2,1)} (2, 3, 4, 6, 7) \xmapsto{(0,-1,1)}$$
$$(1, 2, 3, 5, 6) \xmapsto{(0.3,-0.8,3)} (1, 2, 2.5, 4.2, 5.2) \xmapsto{(0,-0.2,5)} \sigma$$

There is an improved cost 3.3 chronological sequence of moves obtained by reordering the above sequence and combining moves at the same position :

$$\gamma \xmapsto{(-1,1,1)} (1, 2, 3, 5, 6) \xmapsto{(0,0,2)} (1, 2, 3, 5, 6) \xmapsto{(0.3,-0.8,3)}$$
$$(1, 2, 2.5, 4.2, 5.2) \xmapsto{(0,0,4)} (1, 2, 2.5, 4.2, 5.2) \xmapsto{(0,-0.2,5)} \sigma$$

There corresponding reverse chronological sequence of moves is of course :

$$\gamma \xmapsto{(0,-0.2,5)} (1,1,2,4,4.8) \xmapsto{(0,0,4)} (1,1,2,4,4.8) \xmapsto{(0.3,-0.8,3)}$$

$$(1,1,1.5,3.2,4) \xmapsto{(0,0,2)} (1,1,1.5,3.2,4) \xmapsto{(-1,1,1)} \sigma$$

As the above example suggests, we claim the following :

Lemma 9. *There is a minimal cost sequence of moves aligning any two linear timed traces that is chronological/reverse chronological.*

Proof. We assume that there is at least one mixed move at each position, adding zero moves as needed. We first observe the following.

$$\forall 1 \leq i < j \leq n : (s,d,i)(s',d',j)(\gamma) = \gamma' = (s',d',j)(s,d,i)(\gamma)$$

Where

$$\forall 1 \leq k \leq n : \gamma'_k = \begin{cases} \gamma_k & k < i \\ \gamma_k + s + d & k = i \\ \gamma_k + d & i < k < j \\ \gamma_k + s' + d + d' & k = j \\ \gamma_k + d + d' & k > j \end{cases}$$

That is, on linear timed traces, mixed moves are commutative. Clearly permuting the moves of a run does not change its cost, so we can assume any sequence of moves to be in increasing order of positions without loss of generality.

Now we can write m' as follows :

$$m' = (s_{11}, d_{11}, 1) \dots (s_{ij}, d_{ij}, i) \dots (s_{nk_n}, d_{nk_n}, n)$$

Where for each i, $k_i \geq 1$ is the number of mixed moves at position i.

$$cost(m') = \sum_{i=i}^{n} (\sum_{j=1}^{k_i} |s_{ij}| + |d_{ij}|)$$

Define $s_i = \sum_{j=1}^{k_i} s_{ij}$, $d_i = \sum_{j=1}^{k_i} d_{ij}$, giving us the chronological run

$$m = (s_1, d_1, 1) \dots (s_i, d_i, i) \dots (s_n, d_n, n)$$

Now, by the triangle inequality we know that

$$\forall i : \sum_{j=1}^{k_i} |s_{ij}| + |d_{ij}| \geq |\sum_{j=1}^{k_i} s_{ij}| + |\sum_{j=1}^{k_i} d_{ij}| = cost(s_i, d_i, i)$$

Thereby giving us that

$$cost(m) \leq cost(m')$$

As for the reverse chronological sequence, by commutativity we can simply reverse m and obtain m_R, a reverse chronological sequence of moves that does the same transformation and as reversing the sequence preserves cost,

$$cost(m_R) = cost(m) \leq cost(m')$$

□

6.2.2 Co-operation

Definition 23 (Co-operation)**.** *A mixed move is said to be co-operative if its delay edit and stamp edit are in the same direction, that is,*

$$(s, d, i) \text{ is co-operative iff } sd \geq 0$$

An ordered sequence of moves $m = m_0 m_1 \ldots m_{n-1}$ aligning γ to σ is said to be co-operative if each of its moves is co-operative.

This property has nothing to do with the words it is trying to align, and as a result, is largely unaffected by whether we're viewing the words in timing or flow representation.

Example 12. The previous example (that was a cost 3.3 run, recall) can hence be further improved to become co-operative cost 1.7 run as follows :

$$\gamma \xmapsto{(0,0,1)} (1,1,2,4,5) \xmapsto{(1,0,2)} (1,2,2,4,5) \xmapsto{(0.5,0,3)}$$

$$(1,2,2.5,4,5) \xmapsto{(0.2,0,4)} (1,2,2.5,4.2,5) \xmapsto{(0,0,5)} \sigma$$

With a reverse chronological counterpart on the flow vector:

$$f_\gamma \xmapsto{(0,0,5)} (1,0,1,2,1) \xmapsto{(0.2,0,4)} (1,0,1,2.2,0.8) \xmapsto{(0.5,0,3)}$$

$$(1,0,1.5,1.7,0.8) \xmapsto{(1,0,2)} (1,1,1.5,1.7,0.8) \xmapsto{(0,0,1)} f_\sigma$$

Once again, as the example suggests, one can convert a non-cooperative run into a cooperative one and improve cost while we're at it.

Lemma 10. *There is a cooperative minimal cost chronological (or reverse chronological) sequence of moves aligning any two linear timed traces.*

Proof. We first prove that there is a co-operative and chronological minimal cost sequence of moves aligning any two linear timed traces. Given any minimal cost sequence of moves aligning γ to σ, by Lemma 9 we know that there is a chronological sequence that is also minimal cost, call it m'.

Assume $|\gamma| = |\sigma| = n$.

We proceed by induction on the first index k at which $m' = (s_1, d_1, 1) \ldots (s_n, d_n, n)$ uses a non co-operative move.

Base case : $k = n$.

This gives us

$$s_n d_n < 0 \implies |s_n + d_n| < |s_n| + |d_n|$$
$$\implies m = m'|_{n-1}(0, s_n + d_n, n)$$

costs less than m', aligns the words and is co-operative.

Induction hypothesis : Suppose we've demonstrated that there is a sequence of moves m'' that aligns γ to σ, is co-operative for the first $k-1 < n-1$ steps, is non co-operative at step k and $cost(m'') \leq cost(m)$

Claim : There is a sequence of moves m that aligns γ to σ, is co-operative for at least the first k steps and

$$cost(m) \leq cost(m'') \leq cost(m')$$

Proof : Let $m'' = (s_1, d_1, 1) \dots (s_n, d_n, n)$.

We first define $m_i = m''|_{k-1}$, $m_f = (s_{k_2}, d_{k+2}, k+2) \dots (s_n, d_n, n)$, thereby giving us

$$m'' = m_i(s_k, d_k, k)(s_{k+1}, d_{k+1}, k+1)m_f$$

We know that $s_k d_k < 0$. This can be split into the following cases :

Case 1 : $|d_k + s_k| = |d_k| - |s_k|$

We define the following aligning, chronological run that is co-operative up to (and including) the kth step :

$$m = m_i(0, d_k + s_k, k)(s_{k+1}, d_{k+1} - s_k, k+1)m_f$$

$$cost(m'') - cost(m)$$
$$= |s_{k+1}| + |d_{k+1}| + |s_k| + |d_k| - (|d_k + s_k| + |s_{k+1}| + |d_{k+1} - s_k|)$$
$$\geq (|s_{k+1}| + |d_{k+1}| - |s_{k+1}| + |d_{k+1}|) + (|s_k| + |d_k| - |d_k + s_k| - |s_k|) \geq 0$$

Case 2 : $|d_k + s_k| = |s_k| - |d_k|$

We define the following aligning, chronological run that is co-operative up to (and including) the kth step :

$$m = m_i(d_k + s_k, 0, k)(s_{k+1}, d_k + d_{k+1}, k+1)m_f$$

$$cost(m'') - cost(m)$$
$$= |s_{k+1}| + |d_{k+1}| + |s_k| + |d_k| - (|s_k + d_k| + |s_{k+1}| + |d_{k+1} + d_k|)$$
$$\geq |d_{k+1}| + |s_k| + |d_k| - (|s_k + d_k| + |d_{k+1}| + |d_k|) \geq 0$$

By induction we can hence conclude that there is a chronological sequence of moves m that is co-operative throughout and has lower cost than m', i.e, if m' is minimal cost so is m.

Now, m_R is also co-operative throughout as co-operation is a property of the individual moves constituting the sequence, and m_R has exactly the same moves just in the reverse order, so there is also a reverse chronological minimal cost sequence of moves aligning γ to σ.

□

Very similar to co-operation is the notion of *cross co-operation*. Much like how mixed moves at a position should work together instead of against each other, a stamp move s at a location should ideally have opposite sign to the delay d at the next position, as its contribution to the next flow component is going to be $-s$. This leads us to the following definition :

Definition 24. *A reverse chronological run $m = (0, d_n, n) \dots (s_1, d_1, 1)$ is said to be cross co-operative if no stamp component has the same sign as the delay component of the move played at the next position, that is,*

$$\forall i < n : s_i \cdot d_{i+1} \leq 0$$

Much like co-operation above, cross co-operation can be used to improve the cost of a run, as seen in the next lemma.

Lemma 11. *For every reverse chronological co-operative run m' aligning γ to σ, there is a reverse chronological, co-operative and cross co-operative run m aligning γ to σ such that*

$$cost(m) \leq cost(m')$$

Proof. Say $m' = (0, d'_n, n) \dots (s'_1, d'_1, 1)$. Let i be an index at which the stamp move is not cross co-operative, i.e, $s_i \cdot d_{i+1} > 0$.

Consider a run m'' that is identical to m', except for the moves it plays at positions i and $i + 1$.

Let m'' perform the move $(0, d_i + s_i, i)$ at position i and $(s_{i+1}, d_{i+1} - s_i, i+1)$ for the move at position $i+1$ if $|s_i| < |d_i|$ and $(s_i - d_{i-1}, d_i + d_{i+1}, i)$ at position i and $(s_{i+1}, 0, i + 1)$ for the move at position $i + 1$ otherwise.

Clearly, the appropriate one of either of these is cross co-operative at position i, while also staying co-operative and reverse chronologically aligning γ to σ, and costs $\min(|s_i|, |d_{i-1}|)$ less that m'.

Doing this procedure for at most n times will result in a run with every stamp move being cross co-operative and with lower cost than m', that is, our required m. □

6.2.3 Stability

The final property we define will be largely used for reverse chronological runs on flow function representations, and it is the last improvement one can make on the cost of a sequence of moves as after enforcing this property, the sequence of moves that results is unique. We stop to prefer this representation for our current needs for two reasons, one is that the reverse chronological setting is more useful for other types of process models where chronological algorithms fail, and the insistence on the flow function representation is because when dealing with reverse chronological models, the flow function still retains the notion of partially aligning suffixes of the traces as we go along, which is quite hard to see as clearly in the timing function representation. Moreover, we define stability in terms of flow values rather than timestamp values because flow vectors and the effects of moves on them can be studied locally, while

timestamp series get perturbed throughout their length by individual moves, and hence trying to optimise for the effect of a single mixed move involves too many variables. Hence, until further notice, consider any co-operative run to be reverse chronological, and definitions and algorithms will favour aligning flow functions rather than timing functions. It is quite clear to see that completely aligning a flow function is equivalent to completely aligning its timing function, so this is acceptable.

The notion of co-operation and reverse chronology leaves only one degree of freedom so to speak in our choice of aligning moves. One say, the stamp components of a sequence of moves has been decided, the delay components are predetermined to align the flow function from right to left.

Now, at any point, a delay move only affects one flow component, and adds to the cost of one mixed move. A stamp move also only adds to the cost of one mixed move, but it has the ability to affect two flow components, hence it is natural to want to choose stamp moves in a manner that corrects both the flow components it belongs to to the best of its abilities, thereby extracting as much use as possible from as little cost, and leaving the rest up to the delay component to correct. It is this reasoning that brings us to the property we define below as stability, which denotes this prudent choice of s.

Definition 25 (Stability)**.** *In a reverse chronological, co-operative sequence of moves, let*

$m_i = (s, d, i)$ *be a co-operative move seeking to correct the partially aligned flow function*

$f_\gamma = (f_\gamma(1), \dots f_\gamma(i), f_\gamma(i+1), f_\sigma(i+2) \dots f_\sigma(n))$ *(the result of the run of m all the way up to the last stamp move) to* $f_\sigma = (f_\sigma(1), f_\sigma(2), \dots f_\sigma(n))$.

Let $e_i = f_\sigma(i) - f_\gamma(i)$, *and* $e_{i+1} = f_\sigma(i+1) - f_\gamma(i+1)$.

We say m_i is stable if

$$s = \begin{cases} 0 & e_i \cdot e_{i+1} \geq 0 \\ e_i & \textit{Else If } |e_i| < |e_{i+1}| \\ -e_{i+1} & \textit{Otherwise} \end{cases}$$

By default that the only stable move at the last position is a pure delay.

A co-operative sequence of moves is said to be *stable* if each of its moves is stable.

Note that any stable run is also cross co-operative.

Lemma 12. *The stable, co-operative, reverse chronological sequence of moves aligning any two linear timed traces is unique.*

Proof. This is clear, as given a partially aligned flow trace the choice of s to make the next move stable as defined above is deterministic, and the corresponding delay moves are determined by reverse chronology and co-operation. □

Example 13. Hence, the earlier cost 1.7 example can be even further improved to be stable and have cost 1.5, thereby giving us the following :

$$f_\gamma \xmapsto{(0,-0.2,5)} (1, 0, 1, 2, 0.8) \xmapsto{(0,-0.3,4)} (1, 0, 1, 1.7, 0.8) \xmapsto{(0,0,3)}$$

$$(1,0,1,1.7,0.8) \xmapsto{(0.5,0.5,2)} (1,1,1.5,1.7,0.8) \xmapsto{(0,0,1)} \sigma$$

Once again, as the example suggests, we claim that stability improves the cost of the sequence of moves, but moreover, a stable chronological run (or reverse chronological run) is unique.

Lemma 13. *There is a stable minimal cost sequence of moves aligning any two linear timed traces.*

Before we proceed with the proof of this result, we prove a small technical lemma that will help us in the proof of Lemma 13. The motivation behind the following result is essentially that the mixed distance d_N respect an intuitive notion of closeness, once two words have been aligned up till the penultimate place, the one whose last duration since firing lands it closer to the desired target will naturally be the easier one to align. Put another way, if two words, aligning to a target word, are identical to each other in all respects but the last timestamp, then the one whose last flow value is closer to the desired target flow value, is the word that is over closer to the target under d_N between the two.

Lemma 14. *Given two timing functions*

$$\gamma^x = (\gamma_1, \gamma_2, \dots \gamma_{n-1}, \gamma_{n-1} + x)$$

and

$$\gamma^y = (\gamma_1, \gamma_2, \dots \gamma_{n-1}, \gamma_{n-1} + y)$$

Both aligning to $\sigma = (\sigma_1, \dots, \sigma_n)$*, such that* $|x-(\sigma_n-\sigma_{n-1})| \leq |y-(\sigma_n-\sigma_{n-1})|$ *then*

$$d_N(\sigma, \gamma^x) \leq d_N(\sigma, \gamma^y)$$

Proof. Consider any run m^y aligning γ^y to σ. We claim there is a run m^x aligning γ^x to σ such that $cost(m^x) \leq cost(m^y)$.

This proves the above lemma as then the minimal cost run aligning γ^x to σ would have cost at most m^x corresponding to the minimal cost run aligning γ^y to σ, so

$$d_N(\sigma, \gamma^x) \leq d_N(\sigma, \gamma^y)$$

Now, we can assume without loss of generality by Lemmas 9 and 10 that m^y is chronological and co-operative, say $m^y = (s_1, d_1, 1) \dots (0, d_n, n)$.

We construct $m^x = (s_1, d_1, 1)(s_2, d_2, 2) \dots (s'_{n-1}, d'_{n-1}, n-1)(0, d'_n, n)$ differing at most at the last two mixed moves. As $m^x|_{n-2} = m^y|_{n-2}$ we can also conclude that $s_{n-1} + d_{n-1} = s'_{n-1} + d'_{n-1}$ by chronology, so with co-operation we can conclude that $cost(m^y) - cost(m^x) = |d_n| - |d'_n|$.

Hence, the problem is reduced to proving that whatever the choice of d_n, there is a choice of d'_n that fully aligns γ^x and has atmost equal magnitude.

Now, $|d_n| = |\sigma_{n-1} + y - \sigma_n|$ as $m^y|_{n-1}$ fully corrected up till the penultimate place, and similarly $|d'_n| = |\sigma_{n-1} + x - \sigma_n|$, hence, by definitions of x and y, we have the result.

□

Now, we can proceed with the proof of Lemma 13.

Proof. Suppose we seek to align the linear timed trace γ to σ, and consider a minimal cost, reverse chronological, co-operative and cross co-operative sequence of moves m' that aligns γ to σ. We know m' exists by Lemmas 10 and 11. On the other hand, consider the unique stable, co-operative, reverse chronological sequence of moves m that aligns γ to σ, that exists by Lemma 12.

We proceed by way of induction. In the base case of words of length 1, the claim is clearly true as the minimal cost aligning move would just be the minimum delay move, which is also the unique stable run. For words of length 2, i.e, $\gamma_1\gamma_2$ aligning to $\sigma_1\sigma_2$, a reverse chronological co-operative run on the flow function is of the following form :

$$m' : f_\gamma \xmapsto{(0,d_2)} (f_\gamma(1), f_\gamma(2) + d_2) \xmapsto{(s_1,d_1)} (f_\gamma(1) + s_1 + d_1, f_\gamma(2) - s_1 + d_1) = f_\sigma$$

By reverse chronology and co-operation, we have the following equation :

$$cost(m') = |d_2| + |s_1 + d_1| = |f_\sigma(2) - f_\gamma(2) + s_1| + |f_\sigma(1) - f_\gamma(1)|$$

Clearly the only variable in the cost is the value of s_1, and the stable run m is the run that precisely minimises the value $|f_\sigma(2) - f_\gamma(2) + s_1|$ while maintaining co-operation, so the claim holds.

Now, let the length of γ be $n > 2$, and suppose the claim holds for all words of length less than n. We claim that $cost(m) \leq cost(m')$.

The two runs must diverge at some point, as otherwise their cost is equal and the claim is obvious. There are only two main cases of interest :

Case 1. The first move is the same, but the second moves are different (i.e the mixed move played at the second right-most position of the trace).

Case 2. The first moves of the two sequences m and m' (i.e the delay played on the right-most position of the trace) are different.

If the first move where the sequences diverge are later, say at some position i in the trace, then we can restrict our attention to the $i+1$ length prefix of the word and the appropriate truncations of m and m', and by induction hypothesis we are done. So, we focus on the above cases henceforth.

Case 1. Let $m = (0, d_n, n)(s_{n-1}, d_{n-1}, n-1)\dots(s_1, d_1, 1)$ and
$m' = (0, d'_n, n)(s'_{n-1}, d'_{n-1}, n-1)\dots(s'_1, d'_1, 1)$

As both these runs are reverse chronological, we obtain the following equalities:

$$\forall i > 1 : s_i + d_i - s_{i-1} = f_\sigma(i) - f_\gamma(i) = s'_i + d'_i + s'_{i-1}$$

Setting i to n gives $d_n - s_{n-1} = d'_n - s'_{n-1} = f_\sigma(n) - f_\gamma(n)$, which implies that since $d_n = d'_n$, $s_{n-1} = s'_{n-1}$.

Now consider $\gamma_s = (s_{n-1}, 0, n-1)\gamma|_{n-1}$, and
$m_s = (0, d_{n-1}, n-1)(s_{n-2}, d_{n-2}, n-2)\dots(s_1, d_1, 1)$ and
$m'_s = (0, d'_{n-1}, n-1)(s'_{n-2}, d'_{n-2}, n-2)\dots(s_1, d_1, 1)$.

Clearly m_s is the stable run aligning γ_s to $\sigma|_{n-1}$ and m'_s is another reverse chronological co-operative aligning sequence, and

$$cost(m) = |d_n| + |s_{n-1}| + cost(m_s)$$

$$cost(m') = |d'_n| + |s'_{n-1}| + cost(m'_s)|$$

and by triangle inequality and the stability of m,

$$|d_n| + |s_{n-1}| = |d_n - s_{n-1}| = |d'_n - s'_{n-1}| \leq |d'_n| + |s_{n-1}|$$

Now, by induction hypothesis on γ_s, m_s, m'_s (they're an instance of length $n-1$ words, case 2), we deduce $cost(m_s) \leq cost(m'_s)$, by which we can conclude that

$$cost(m) \leq cost(m')$$

Case 2. Let $m = (0, d_n, n)(s_{n-1}, d_{n-1}, n-1)\dots(s_1, d_1, 1)$ and $m' = (0, d'_n, n)(s'_{n-1}, d'_{n-1}, n-1)\dots(s'_1, d'_1, 1)$

As both these runs are reverse chronological, we obtain the following equalities:

$$\forall i > 1 : s_i + d_i - s_{i-1} = f_\sigma(i) - f_\gamma(i) = s'_i + d'_i + s'_{i-1}$$

Setting i to n gives $d_n - s_{n-1} = d'_n - s'_{n-1} = f_\sigma(n) - f_\gamma(n)$, which implies that having performed the first delay and stamp move of each run, one gets the following :

$$f_\gamma \xmapsto{(0,d,n)(s,0,n-1)} (f_\gamma(1), \dots f_\gamma(n-1) + s, f_\sigma(n))$$

$$f_\gamma \xmapsto{(0,d',n)(s',0,n-1)} (f_\gamma(1), \dots f_\gamma(n-1) + s', f_\sigma(n))$$

Where $s = s_{n-1}$, $d = d_n$, $s' = s'_{n-1}$, and $d' = d'_n$.

Now, let $e_n = f_\sigma(n) - f_\gamma(n)$, and $e_{n-1} = f_\sigma(n-1) - f_\gamma(n-1)$ Now, by stability of m, we have the following definition of s :

$$s = \begin{cases} 0 & e_n \cdot e_{n-1} \geq 0 \\ e_{n-1} & \text{Else If } |e_{n-1}| < |e_n| \\ -e_n & \text{Otherwise} \end{cases}$$

By the above definition, we see that $s = \text{argmin}_{x \cdot (e_n - x) \geq 0} |e_{n-1} - x|$.

Now by cross co-operation, we know that $s' \cdot (e_n - s') \geq 0$, so this means

$$|e_{n-1} - s| \leq |e_{n-1} - s'|$$

$$\implies |f_\sigma(n-1) - (f_\gamma(n-1) + s)| \leq |f_\sigma(n-1) - (f_\gamma(n-1) + s')|$$

Now, consider the words $\gamma_s = (\gamma_1, \dots \gamma_{n-2}, \gamma_{n-1} + s)$ and $\gamma'_s = (\gamma_1, \dots \gamma_{n-2}, \gamma_{n-1} + s')$ both aligning to $\sigma|_{n-1}$. We can apply Lemma 14 to these, and deduce that

$$d_N(\gamma_s, \sigma|_{n-1}) \leq d_N(\gamma'_s, \sigma|_{n-1})$$

Now, by induction hypothesis, as the rest of m would have been the stable minimal cost run aligning γ_s to $\sigma|_{n-1}$, along with stability of m, we have the following :

$$cost(m) = |s| + |d| + d_N(\gamma_s, \sigma|_{n-1}) \leq |d - s| + d_N(\gamma'_s, \sigma|_{n-1}) \leq cost(m')$$

And hence we are done.

□

Algorithm 6 d_N Backwards Computation Algorithm

```
Input :  σ, γ
Output :  d_N(γ, σ)
cost ← 0
i ← n
while i > 1 do
    a ← f_σ(i) − f_γ(i)
    b ← f_σ(i − 1) − f_γ(i − 1)
    if a · b ≥ 0 then                                  ▷ (0, a, i)
        γ ← (0, a, i)γ
    else if |a| < |b| then                             ▷ (−a, 0, i − 1)(0, 0, i)
        γ ← (−a, 0, i − 1)γ
    else                                               ▷ (b, 0, i − 1)(0, a − b, i)
        γ ← (b, 0, i − 1)(0, a − b, i)γ
    end if
    cost ← cost + |a|
    i ← i − 1
end while
cost ← cost + |γ_1 − σ_1|                              ▷ (0, |σ_1 − γ_1|, 1)
```

6.3 Algorithms

Lemma 15. *Given two time sequences $\sigma = (\sigma_1, \sigma_2, \ldots \sigma_n)$ and $\gamma = (\gamma_1, \gamma_2, \ldots, \gamma_n)$ with linear underlying causal processes, the sequence of moves the above algorithm calculates (m) corresponds to the unique stable sequence of moves that aligns σ to γ.*

Proof. This is clear by the definition of stability. □

Theorem 16. *Algorithm 6 is correct, that is, its result (cost) $= d_N(\gamma, \sigma)$*

Proof. This holds as by Lemma 15 the algorithm calculates the unique stable sequence of aligning moves, and by Lemma 13 this must be the minimal cost sequence. □

Theorem 17. *Given a linear causal process (CN, p) of a time Petri Net N and a linear observed trace σ, the word $\gamma \in \mathcal{L}(N)$ such that $\gamma = \underset{x \in \mathcal{L}(N)}{\operatorname{argmin}}\ d_\theta(x, \sigma)$ also has the property $\gamma = \underset{x \in \mathcal{L}(N)}{\operatorname{argmin}}\ d_N(x, \sigma)$.*

Proof. As (CN, p) is linear, we can consider the event set E to be totally ordered by G, giving us the list $\{e_1, e_2, \ldots e_n\}$ and their respective static interval constraints $\forall i \leq n : SI(p(e_i)) = [a_i, b_i]$. In this setting, clearly,

$$f_\gamma(i) = \underset{x \in [a_i, b_i]}{\operatorname{argmin}} |x - f_\sigma(i)|$$

We claim that for all $\alpha \in \mathcal{L}(N)$ i.e $\forall \alpha : f_\alpha(i) \in [a_i, b_i]$,

$$d_N(\sigma, \gamma) \leq d_N(\sigma, \alpha)$$

By Theorem 16 we shall henceforth implicitly equate the cost of the stable run aligning a word to the mixed moves distance to the word. Let m, m' be the stable runs aligning γ, α respectively to σ, where:

$$m = (0, d_n, n)(s_{n-1}, d_{n-1}, n-1) \dots (s_1, d_1, 1)$$

$$m' = (0, d'_n, n)(s'_{n-1}, d'_{n-1}, n-1) \dots (s'_1, d'_1, 1)$$

In addition, we know that $cost(m) = \sum_{i=1}^{n}(|s_i| + |d_i|)$, and $cost(m') = \sum_{i=1}^{n}(|s'_i| + |d'_i|)$.

We break the run up after each stamp move, but not delay move, at a particular position has been performed, as below:

$$\forall i > 0 : c_i = |s_{i-1} + |d_i|, c'_i = |s'_{i-1}| + |d'_i|$$

Where we let $s_0 = s'_0 = 0$ for notation's sake. Clearly $cost(m) = \sum_{i=1}^{n} c_i$ and $cost(m') = \sum_{i=1}^{n} c'_i$

We wish to prove that

$$cost(m) \leq cost(m')$$

In fact, we shall prove something stronger. We define the notion of the disadvantage of m, which denotes in effect the amount of distance it might have to catch up in aligning due to m' having done a costlier move just prior.

$$dis(i) = max(0, c'_i - c_i)$$

Note of course, that each c_i, c'_i and $dis(i)$ is nonnegative.

Now, we claim the following :

$$\forall k > 0 : \sum_{i=k}^{n} c_i + dis(k) \leq \sum_{i=k}^{n} c'_i$$

If we prove this, we will be done, as the equation for $k = 1$ proves that m has overall lower cost than m'.

Let us proceed by induction on $n - k$.

For the base case, $k = n$, so the equation we seek is that $c_n + dis(n) \leq c'_n$, which is clear as $dis(n) = max(0, c'_n - c_n)$.

Now, say the claim holds for all $k > K$ for some $K < n$. We seek to now prove that

$$c_K + \sum_{i=K+1}^{n} c_i + dis(K) \leq c'_K + \sum_{i=K+1}^{n} c'_i$$

Now, if $c_K \leq c'_K$ this is clear as $c_K + dis(K) = max(c_K, c'_K)$. On the other hand, suppose $c_K > c'_K$.

By the induction hypothesis, we know that

$$\sum_{i=K+1}^{n} c_i + dis(K+1) \leq \sum_{i=K+1}^{n} c'_i$$

So if we show that $c_K - c'_K \leq dis(K+1)$, we are done.

Now, by reverse chronology, co-operation and cross co-operation of m and m', and the definition of γ, we know the following inequality holds :

$$|s_K| + c_K = |s_K + d_K - s_{K-1}| = |f_\sigma(K) - f_\gamma(K)|$$

$$\leq |f_\sigma(K) - f_\alpha(K)| = |s'_K + d'_K - s'_{K-1}| = |s'_K| + c'_K$$

So, $c_K > c'_K \implies |s_K| < |s'_K|$, and moreover,

$$c_K - c'_K \leq |s'_K - s_K|$$

Now, by stability of m, we know that s_K was defined as below :

$$s_K = \begin{cases} 0 & e_K \cdot e_{K+1} \geq 0 \\ e_K & \text{Else If } |e_K| < |e_{K+1}| \\ e_{K+1} & \text{Otherwise} \end{cases}$$

Where $e_K = (f_\sigma(K) - f_\gamma(K))$ and $e_{K+1} = f_\sigma(K+1) - f_\gamma(K+1) - s_{K+1}$.

We proceed to analyse the possibilities based on the various possible stable choices of s_K depending on the state of the run so far.

Suppose firstly that $s_K = 0$. There are a number of ways this could hold. By choice of γ,

$$(f_\sigma(K) - f_\gamma(K)) \cdot (f_\sigma(K+1) - f_\gamma(K+1)) > 0$$

$$\implies (f_\sigma(K) - f_\alpha(K)) \cdot (f_\sigma(K+1) - f_\alpha(K+1)) > 0 \implies s'_K = 0$$

But $|s'_K| > |s_K| \geq 0$ so this first subcase is not possible.

Secondly, if

$$(f_\sigma(K) - f_\gamma(K)) = 0 \implies c_K = 0 > c'_K$$

Which is also not possible.

Thirdly, if

$$(f_\sigma(K+1) - f_\gamma(K+1) - s_{K+1} = 0 \implies d_{K+1} = 0$$

Now, $d_{K+1} = 0$ and $|s'_K| > |s_K|$ implies

$$dis(K+1) = |s'_K| + |d'_{K+1}| - |s_K| \geq |s'_K| - |s_K| \geq c_K - c'_K$$

So in this case the claim holds as needed.

Otherwise, suppose $s_K = f_\sigma(K) - f_\gamma(K)$. In this case, c_K is once again equal to zero, which is not possible as by assumption $c_K > c'_K \geq 0$.

Now, the last possibility is that $s_K = f_\gamma(K+1) - f_\sigma(K+1) - s_{K+1}$. This means that once s_K is played, the $K+1$ position is perfectly aligned, so $d_{K+1} = 0$. Once again, this means

$$dis(K+1) = |s'_K| + |d'_{K+1}| - |s_K| \geq |s'_K| - |s_K| \geq c_K - c'_K$$

So in this case also, the claim holds as needed.

Hence, we see that in every possible case, the claim holds for $k = K$ as well, so by induction we see that

$$\sum_{i=1}^{n} c_i + dis(K+1) \leq \sum_{i=1}^{n} c'_i$$

From which we can conclude that

$$cost(m) = \sum_{i=1}^{n} c_i) \leq \sum_{i=1}^{n} c'_i = cost(m')$$

Hence, the result holds, and γ is an optimal word of $\mathcal{L}(N)$ to align σ to.

□

This of course means that we have a linear time algorithm for aligning linear causal processes under d_N, by locally choosing the best flow vector as above.

7 Relating timed process models back to BPMN

An important thing to keep in mind is that in practice, the business process mining community does not use Petri nets for their process models, but a similar directed graph representation known as BPMN or business process mining notation.

Example 14. example of BPMN

The question of converting one to the other in the untimed case has been well studied, as for formal analysis Petri nets are always preferred but for real life applications BPMN is highly convenient and expressive. (cite)

So a practical consideration of interest is whether time Petri nets can be expressed in BPMN as well, and how to translate between the two. Thankfully, BPMN supports a construct known as a timer event, which comes in three principal forms (list) (cite)

These can be seen as absolute and relative constraints that are expressible in time Petri nets as (something).

8 Implementation

9 Perspectives and Conclusion

References